

```

*****
#*
#* Copyright (c) 2026 BlackMaxMetal
#*
#* This program is free software; you can redistribute it and/or modify
#* it under the terms of the GNU Lesser General Public License (LGPL)
#* as published by the Free Software Foundation; either version 2 of
#* the License, or (at your option) any later version.
#* for detail see the LICENCE text file.
#*
#* This program is distributed in the hope that it will be useful,
#* but WITHOUT ANY WARRANTY; without even the implied warranty of
#* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
#* GNU Library General Public License for more details.
#*
#* You should have received a copy of the GNU Library General Public
#* License along with this program; if not, write to the Free Software
#* Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307
#* USA
#*
*****

```

```

import FreeCAD
import FreeCADGui
# Use unified PySide wrapper for cross-version stability in FreeCAD 26.3
from PySide import QtWidgets, QtCore

# --- SPREADSHEET REVERSIBLE WINDOW LAYER ---
class DetachedSpreadsheetWindow(QtWidgets.QMainWindow):

    def __init__(self, sheet_widget, mdi_subwin, notice_label, doc_name,
filter_instance):
        super(DetachedSpreadsheetWindow, self).__init__(None, QtCore.Qt.Window |
QtCore.Qt.CustomizeWindowHint | QtCore.Qt.WindowMinMaxButtonsHint |
QtCore.Qt.WindowCloseButtonHint)
        self.sheet_widget = sheet_widget
        self.mdi_subwin = mdi_subwin
        self.notice_label = notice_label
        self.target_doc_name = doc_name
        self.filter_instance = filter_instance

        self.setWindowTitle(f"Detached Spreadsheet - {mdi_subwin.windowTitle()}")
        self.setStyleSheet("QMainWindow { background-color: #2b2b2b; }")
        self.setCentralWidget(sheet_widget)

        # --- SPREADSHEET TOOLBAR CLONING INJECTION ---
        # Locate and inject relevant spreadsheet actions into this secondary window
layout
        self._inject_spreadsheet_toolbar()

        if sheet_widget:
            sheet_widget.show()

        # Preserved native layout execution call
        self.show()

    def _inject_spreadsheet_toolbar(self):
        """Locates FreeCAD's native spreadsheet toolbar actions specifically,
avoiding other sheets-based workbenches."""

```

```

try:
    mw = FreeCADGui.getMainWindow()
    if not mw:
        return

    # Create a clean external toolbar container on this secondary
    QMainWindow instance
    external_tb = self.addToolBar("Spreadsheet Tools")
    external_tb.setMovable(False)
    external_tb.setStyleSheet("QToolBar { background-color: #333333;
border: 1px solid #555555; padding: 2px; }")

    # Strict helper function to scan application toolbars without trapping
    SheetMetal or other plugins
    def find_and_copy_spreadsheet_toolbar():
        for tb in mw.findChildren(QtWidgets.QToolBar):
            tb_name = tb.objectName().lower()
            # STRICT CHECK: Block third-party sheetmetal items while
            targetting only the native spreadsheet core
            if "spreadsheet" in tb_name and "metal" not in tb_name:
                for action in tb.actions():
                    if action.isSeparator():
                        external_tb.addSeparator()
                    else:
                        external_tb.addAction(action)
                return True
        return False

    # First attempt: Verify if the official spreadsheet toolbar is already
    drawn on screen
    found_actions = find_and_copy_spreadsheet_toolbar()

    # FALLBACK INITIALIZATION: If unavailable, force FreeCAD to initialize
    the actual Spreadsheet environment
    if not found_actions:
        try:
            # Request FreeCAD command framework to forcefully wake and
            structure the Spreadsheet workbench layout
            FreeCADGui.activateWorkbench("SpreadsheetWorkbench")
            QtWidgets.QApplication.processEvents()
            # Second attempt: check if the targeted toolbar is now
            initialized and exposed
            found_actions = find_and_copy_spreadsheet_toolbar()
        except Exception:
            pass

    # Ultimate secure layout fallback: manually build and inject core
    formatting actions if toolbar remains hidden
    if not found_actions:
        # Array of targeted command tokens registered into FreeCAD's core
        registry actions system
        core_cmds = [
            "Spreadsheet_AlignLeft", "Spreadsheet_AlignCenter",
            "Spreadsheet_AlignRight",
            "Spreadsheet_StyleBold", "Spreadsheet_StyleItalic",
            "Spreadsheet_StyleUnderline"
        ]
        for cmd_name in core_cmds:
            cmd = FreeCADGui.getCommand(cmd_name)

```

```

        if cmd:
            # Scan all available actions within the application to link
the command trigger pointer
            for tb in mw.findChildren(QtWidgets.QToolBar):
                for act in tb.actions():
                    if cmd_name in act.objectName() or (act.text() and
cmd_name in act.text()):
                        external_tb.addAction(act)
            except Exception as e:
                FreeCAD.Console.PrintWarning(f"MultiMonitor Toolbar Notice (Spreadsheet
toolbar bypass): {str(e)}\n")

    def closeEvent(self, event):
        """Intercepts window closure to safely return the spreadsheet widget to
FreeCAD native layout before frame destruction."""
        try:
            # 1. Clean up centralized global menu inhibition tracking array
            if hasattr(self, "target_doc_name") and self.target_doc_name:
                if hasattr(FreeCADGui, '_mm_disabled_sheets') and
FreeCADGui._mm_disabled_sheets:
                    FreeCADGui._mm_disabled_sheets.remove(self.target_doc_name)

            # 2. SAFE WIDGET GRAFTING ROLLBACK
            # We evacuate the spreadsheet widget from our external window and
restore it back into FreeCAD's native MDI tab
            if self.sheet_widget and self.mdi_subwin:
                try:
                    self.setCentralWidget(None)
                    self.sheet_widget.setParent(self.mdi_subwin)
                    self.mdi_subwin.setWidget(self.sheet_widget)

                    # Prevent execution loops by disconnecting the destruction
listener temporarily
                    try:
                        self.mdi_subwin.destroyed.disconnect()
                    except Exception:
                        pass

                    # Safely close only the respective native spreadsheet MDI tab
inside FreeCAD
                    self.mdi_subwin.close()
                except Exception:
                    pass

                QtWidgets.QApplication.processEvents()
            except Exception:
                pass

            # 3. Unregister reference pointer from global tracking array memory
structures
            if hasattr(FreeCADGui, '_mm_standalone_clones') and
FreeCADGui._mm_standalone_clones:
                try:
                    if self in FreeCADGui._mm_standalone_clones:
                        FreeCADGui._mm_standalone_clones.remove(self)
                except Exception:
                    pass
            event.accept()

```

```

class TabContextMenuFilter(QtCore.QObject):

    def __init__(self, tab_bar, main_addon_window_class):
        super(TabContextMenuFilter, self).__init__(tab_bar)
        self.tab_bar = tab_bar
        self.window_class = main_addon_window_class

        # Keep track of active document names that currently have an extracted
spreadsheet
        self._disabled_sheets = []

        mw = FreeCADGui.getMainWindow()
        self.mdi_area = mw.findChild(QtWidgets.QMdiArea) if mw else None
        if self.mdi_area:

self.mdi_area.subWindowActivated.connect(self._handle_mdi_window_change)

        # Force FreeCAD's native TabBar to broadcast right-clicks externally
        if self.tab_bar:
            self.tab_bar.setContextMenuPolicy(QtCore.Qt.CustomContextMenu)

self.tab_bar.customContextMenuRequested.connect(self._handle_forced_context_menu)
        #
        # # Instantiate and register our document observer into FreeCAD's native
GUI notification system
        # self._doc_observer = MMDocumentObserver(self)
        # FreeCADGui.addObserver(self._doc_observer)

    def _handle_mdi_window_change(self, active_subwin):
        """Monitors MDI state changes and cross-references active views to force-
close orphaning clones."""
        try:
            current_docs = [doc.Name for doc in FreeCAD.listDocuments().values()]

            if hasattr(FreeCADGui, '_mm_standalone_clones') and
FreeCADGui._mm_standalone_clones:
                for win in list(FreeCADGui._mm_standalone_clones):
                    if win:
                        try:
                            win_doc_name = getattr(win, "target_doc_name", None)
                            if win_doc_name and win_doc_name not in current_docs:
                                win.close()
                                if win in FreeCADGui._mm_standalone_clones:
                                    FreeCADGui._mm_standalone_clones.remove(win)
                                continue
                        except Exception:
                            pass
                    except Exception:
                        pass

        def _handle_forced_context_menu(self, local_pos):
            """Slots directly into Qt's forced context signal to determine tabs
positions and launch actions injection."""
            if not self.tab_bar:
                return
            tab_index = self.tab_bar.tabAt(local_pos)
            if tab_index != -1:
                self._last_tab_rect = self.tab_bar.tabRect(tab_index)

```

```

        # Shift FreeCAD's active workspace target to the tab under the user's
cursor pointer
        self.tab_bar.setCurrentIndex(tab_index)

        # Force FreeCAD to synchronize its active view pointer instantly
        target_sub_win = self.mdi_area.subWindowList()[tab_index]
        if target_sub_win:
            self.mdi_area.setActiveSubWindow(target_sub_win)

        QtWidgets.QApplication.processEvents()
        self.inject_clone_action(tab_index)

    def eventFilter(self, obj, event):
        # Fallback safety layer to let FreeCAD handle standard tabs dragging
behaviors naturally
        return super(TabContextMenuFilter, self).eventFilter(obj, event)

    def _force_close_clones_by_name(self, doc_name):
        """Forcefully destroys external cloned views matching the target document
name to unlock C++ memory."""
        if not doc_name:
            return
        if hasattr(FreeCADGui, '_mm_standalone_clones') and
FreeCADGui._mm_standalone_clones:
            to_remove = []
            # Normalize target token name string to avoid mismatches due to case
casing differences
            target_token = str(doc_name).strip().lower()

            for win in list(FreeCADGui._mm_standalone_clones):
                try:
                    if win:
                        # Extract cached target tags identifiers properties strings
safely
                        win_doc_name = getattr(win, "target_doc_name", None)
                        win_title = win.windowTitle().lower() if hasattr(win,
"windowTitle") else ""

                        # Aggressive cross-referencing matching conditions block to
trap all orphaned canvases
                        is_match = False
                        if win_doc_name and str(win_doc_name).strip().lower() ==
target_token:
                            is_match = True
                        elif target_token in win_title or target_token.replace("_",
" ") in win_title:
                            is_match = True

                        if is_match:
                            # Break memory dependencies links to avoid triggering
nested Qt delete exceptions loops
                            if hasattr(win, "setCentralWidget"):
                                win.setCentralWidget(None)
                                win.setParent(None)
                                win.close()
                                win.deleteLater()
                                to_remove.append(win)
                except Exception:
                    pass

```

```

safely        # Clean up global standalone listing array memory references pointers
               for win in to_remove:
                   if win in FreeCADGui._mm_standalone_clones:
                       FreeCADGui._mm_standalone_clones.remove(win)

def inject_clone_action(self, tab_index):
    """Generates a clean contextual menu containing native FreeCAD actions,
    Spreadsheets, and TechDraw support."""
    if not self.mdi_area or tab_index >= len(self.mdi_area.subWindowList()):
        return

    QtWidgets.QApplication.processEvents()

    native_menu = QtWidgets.QMenu(self.tab_bar)
    native_menu.setAttribute(QtCore.Qt.WA_DeleteOnClose)

    target_sub_win = self.mdi_area.subWindowList()[tab_index]
    widget_item = target_sub_win.widget()
    if not widget_item:
        return

    # --- RESTORE NATIVE FREECAD ACTIONS ---
    close_action = native_menu.addAction("Close")
    close_action.setShortcut("Ctrl+F4")
    close_action.triggered.connect(lambda: target_sub_win.close())

    close_all_action = native_menu.addAction("Close All")
    close_all_action.triggered.connect(lambda:
self.mdi_area.closeAllSubWindows())

    native_menu.addSeparator()

    # --- ADVANCED TYPING CASCADE SCAN ---
    class_name = widget_item.metaObject().className().lower()

    is_3d_view = "view3dinventor" in class_name
    is_spreadsheet = "spreadsheet" in class_name or
widget_item.findChild(QtWidgets.QTableView) is not None
    is_techdraw = "techdraw" in class_name or "qgraphicsview" in class_name

    if is_3d_view:
        clone_action = native_menu.addAction("Open 3D View on Second Monitor")
        clone_action.triggered.connect(lambda:
self.clone_in_new_standalone_window(tab_index))

        quad_action = native_menu.addAction("Open Four View on Second Monitor")
        quad_action.triggered.connect(lambda:
self.clone_in_four_view_window(tab_index))

    elif is_spreadsheet:
        sheet_action = native_menu.addAction("Open Spreadsheet on Second
Monitor")
        sheet_action.triggered.connect(lambda:
self.detach_spreadsheet_window(tab_index))

        if hasattr(FreeCADGui, '_mm_disabled_sheets') and
FreeCADGui._mm_disabled_sheets:

```

```

        gui_doc = FreeCADGui.activeDocument()
        if gui_doc and gui_doc.Document.Name in
FreeCADGui._mm_disabled_sheets:
            sheet_action.setEnabled(False)

    elif is_techdraw:
        td_action = native_menu.addAction("Open TechDraw Page on Second
Monitor")
        td_action.triggered.connect(lambda:
self.detach_techdraw_window(tab_index))

    else:
        neutral_action = native_menu.addAction(f"Active Tab View:
{target_sub_win.windowTitle()}")
        neutral_action.setEnabled(False)

    # --- GEOMETRIC POSITIONING OVERRIDE ---
    tab_rect = getattr(self, '_last_tab_rect', QtCore.QRect(0, 0, 100, 25))
    global_tab_top_left = self.tab_bar.mapToGlobal(tab_rect.topLeft())
    menu_height = native_menu.sizeHint().height()

    final_x = global_tab_top_left.x()
    final_y = global_tab_top_left.y() - menu_height - 2

    native_menu.exec_(QtCore.QPoint(final_x, max(10, final_y)))

def _force_purge_tab_by_subwindow(self, mdi_subwin):
    """Neutral fallback wrapper targeting legacy windows tab evictions routines
safely."""
    if mdi_subwin:
        try:
            mdi_subwin.close()
        except Exception:
            pass

def clone_in_new_standalone_window(self, tab_index):
    """Creates a twin 3D View inside FreeCAD first, then schedules its
extraction safely."""
    if not self.mdi_area or tab_index >= len(self.mdi_area.subWindowList()):
        return
    target_sub_win = self.mdi_area.subWindowList()[tab_index]
    target_sub_win.setFocus()

    gui_doc = FreeCADGui.activeDocument()
    if not gui_doc:
        return
    try:
        # 1. Take a snapshot of the sub-windows list before creating the new
view
        sub_windows_before = self.mdi_area.subWindowList()

        # 2. Tell FreeCAD to create the 3D View inside its native environment
naturally
        gui_doc.createView("Gui::View3DInventor")
        QtWidgets.QApplication.processEvents()

        # 3. Schedule the detachment routine AFTER FreeCAD has finished
initializing the view
        # This tiny delay prevents the C++ engine from crashing during the

```

```

extraction step
    QtCore.QTimer.singleShot(50, lambda:
self._execute_view_extraction(target_sub_win, gui_doc, sub_windows_before))
    except Exception as e:
        FreeCAD.Console.PrintError(f"MultiMonitor Error (Failed single clone
trigger): {str(e)}\n")

    def _execute_view_extraction(self, target_sub_win, gui_doc,
sub_windows_before):
        """Safely extracts the single 3D view and binds its closing lifecycle
directly to the native MDI sub-window."""
        try:
            mw = FreeCADGui.getMainWindow()
            fresh_sub_windows = self.mdi_area.subWindowList()

            if fresh_sub_windows and len(fresh_sub_windows) >
len(sub_windows_before):
                new_view_subwin = fresh_sub_windows[-1]

                single_frame = QtWidgets.QMainWindow(None, QtCore.Qt.Window |
QtCore.Qt.CustomizeWindowHint | QtCore.Qt.WindowMinMaxButtonsHint |
QtCore.Qt.WindowCloseButtonHint)
                single_frame.setWindowTitle(f"3D Clone View -
{target_sub_win.windowTitle()}")
                single_frame.setStyleSheet("QMainWindow { background-color:
#2b2b2b; }")

                view_widget = new_view_subwin.widget()
                if view_widget:
                    new_view_subwin.setWidget(None)
                    single_frame.setCentralWidget(view_widget)
                    view_widget.show()

                new_view_subwin.close()

                # Cache document name string safely before any destruction sequence
alters the pointers
                doc_name = gui_doc.Document.Name

                # --- ADVANCED LIFECYCLE LINK ---
                # When the native window tab is closed, shut down the clone and
force FreeCAD C++ core to unload the document
                def perform_complete_cleanup():
                    try:
                        single_frame.close()
                        if doc_name in FreeCAD.listDocuments():
                            FreeCAD.closeDocument(doc_name)
                    except Exception:
                        pass

                target_sub_win.destroyed.connect(perform_complete_cleanup)

                if not hasattr(FreeCADGui, '_mm_standalone_clones'):
                    FreeCADGui._mm_standalone_clones = []
                    FreeCADGui._mm_standalone_clones.append(single_frame)

                single_frame.showNormal()
                self._apply_monitor_and_geometry(single_frame, mw, 850, 600)
                single_frame.raise_()

```



```

        single_frame.activateWindow()

        if target_sub_win:
            self.mdi_area.setActiveSubWindow(target_sub_win)
    except Exception as e:
        FreeCAD.Console.PrintError(f"MultiMonitor Error (Failed single clone
extraction): {str(e)}\n")

    def clone_in_four_view_window(self, tab_index):
        """Creates an independent container window splitting 4 automated viewpoint
clones into a 2x2 grid."""
        mw = FreeCADGui.getMainWindow()
        if not self.mdi_area or tab_index >= len(self.mdi_area.subWindowList()):
            return
        target_sub_win = self.mdi_area.subWindowList()[tab_index]
        target_sub_win.setFocus()
        gui_doc = FreeCADGui.activeDocument()
        if not gui_doc:
            return
        try:
            QtCore.QTimer.singleShot(0, lambda:
self._execute_four_view_extraction(self.mdi_area, target_sub_win, gui_doc, mw))
        except Exception as e:
            FreeCAD.Console.PrintError(f"MultiMonitor Error (Failed quad clone
trigger): {str(e)}\n")

    def _execute_four_view_extraction(self, mdi_area, target_sub_win, gui_doc, mw):
        """Assembles the 2x2 grid layout and binds its closing lifecycle directly
to the native active MDI sub-window."""
        try:
            quad_frame = QtWidgets.QMainWindow(None, QtCore.Qt.Window |
QtCore.Qt.CustomizeWindowHint | QtCore.Qt.WindowMinMaxButtonsHint |
QtCore.Qt.WindowCloseButtonHint)
            quad_frame.setWindowTitle(f"Four View Matrix Clone (TOP, FRONT, LEFT,
ISOMETRIC) - {target_sub_win.windowTitle()}")
            quad_frame.setStyleSheet("QMainWindow { background-color: #2b2b2b; }")

            # Cache document name string safely before any destruction sequence
alters the pointers
            doc_name = gui_doc.Document.Name

            # --- ADVANCED LIFECYCLE LINK ---
            # When the native window tab is closed, shut down the quad frame and
force FreeCAD C++ core to unload the document
            def perform_quad_cleanup():
                try:
                    quad_frame.close()
                    if doc_name in FreeCAD.listDocuments():
                        FreeCAD.closeDocument(doc_name)
                except Exception:
                    pass

            target_sub_win.destroyed.connect(perform_quad_cleanup)

            central_widget = QtWidgets.QWidget(quad_frame)
            grid_layout = QtWidgets.QGridLayout(central_widget)
            grid_layout.setContentsMargins(4, 4, 4, 4)
            grid_layout.setSpacing(6)
            quad_frame.setCentralWidget(central_widget)

```

```

        view_configs = [(0, 0, "TOP"), (0, 1, "LEFT"), (1, 0, "FRONT"), (1, 1,
"ISOMETRIC")]
        main_active_view = FreeCADGui.activeDocument().ActiveView
        if main_active_view and hasattr(main_active_view, 'saveOrientation'):
            main_active_view.saveOrientation()

        for row, col, orientation in view_configs:
            if main_active_view:
                if orientation == "TOP": main_active_view.viewTop()
                elif orientation == "LEFT": main_active_view.viewLeft()
                elif orientation == "FRONT": main_active_view.viewFront()
                elif orientation == "ISOMETRIC":
main_active_view.viewAxonometric()
                main_active_view.fitAll()
                QtWidgets.QApplication.processEvents()

                current_stack = mdi_area.subWindowList()
                gui_doc.createView("Gui::View3DInventor")
                updated_stack = mdi_area.subWindowList()

                if updated_stack and len(updated_stack) > len(current_stack):
                    cloned_subwin = updated_stack[-1]
                    cloned_subwin.setWindowFlags(QtCore.Qt.Widget |
QtCore.Qt.FramelessWindowHint | QtCore.Qt.CustomizeWindowHint)
                    cloned_subwin.setWindowTitle(f"=== {orientation} VIEW ===")
                    grid_layout.addWidget(cloned_subwin, row, col)
                    cloned_subwin.show()

                if main_active_view and hasattr(main_active_view,
'restoreOrientation'):
                    main_active_view.restoreOrientation()
                    main_active_view.fitAll()

                if not hasattr(FreeCADGui, '_mm_standalone_clones'):
                    FreeCADGui._mm_standalone_clones = []
                    FreeCADGui._mm_standalone_clones.append(quad_frame)
                    quad_frame.showNormal()
                    self._apply_monitor_and_geometry(quad_frame, mw, 1200, 850)
                    quad_frame.raise_()
                    quad_frame.activateWindow()
            except Exception as e:
                FreeCAD.Console.PrintError(f"MultiMonitor Error (Failed quad clone
assembly): {str(e)}\n")

        def detach_spreadsheet_window(self, tab_index):
            """Extracts the native spreadsheet widget, injects a text notice mask, and
handles clean workspace updates."""
            if not self.mdi_area or tab_index >= len(self.mdi_area.subWindowList()):
                return

            target_sub_win = self.mdi_area.subWindowList()[tab_index]
            target_sub_win.setFocus()

            gui_doc = FreeCADGui.activeDocument()
            if not gui_doc:
                return

            try:

```

```

# Fetch the main widget profile mapped into FreeCAD's active sub-window
interface layout
    sheet_widget = target_sub_win.widget()
    if not sheet_widget:
        return

    doc_name = gui_doc.Document.Name

    # --- SAFE EXTRACTION WITH ENVELOPE RESTORATION ---
    # Instantiate our specialized separate window container passing the
original spreadsheet widget
    sheet_frame = DetachedSpreadsheetWindow(sheet_widget, target_sub_win,
None, doc_name, self)
    sheet_frame.target_doc_name = doc_name

    # Create an elegant temporary layout placeholder to populate FreeCAD's
native workspace tab area
    placeholder_card = QtWidgets.QWidget()
    placeholder_card.setStyleSheet("background-color: #2b2b2b;")
    card_layout = QtWidgets.QVBoxLayout(placeholder_card)

    # Injected upscaled monitor icon with HTML span tags safely
    lbl_notice = QtWidgets.QLabel("<span style='font-size:
36px;'>🖥️</span><br><br>The spreadsheet has been moved to an external window.",
placeholder_card)
    lbl_notice.setAlignment(QtCore.Qt.AlignCenter)
    lbl_notice.setStyleSheet("color: #aaaaaa; font-size: 13px; font-weight:
bold; line-height: 150%;")
    card_layout.addWidget(lbl_notice)

    # Assign the temporary text card placeholder into FreeCAD's open MDI
sub-window framework
    target_sub_win.setWidget(placeholder_card)

    # Assign the notice label pointer onto the frame object tracking memory
structures for clean deletions
    sheet_frame.notice_label = placeholder_card

    # Disconnect any old conflicting listeners to avoid race conditions
loops
    try:
        target_sub_win.destroyed.disconnect()
    except Exception:
        pass

    # --- SAFE DOCUMENT CLOSURE ROLLBACK HOOK ---
    # If the user closes the main document directly, we return the widget
home right before deletion
    def restore_on_document_delete():
        try:
            sheet_frame.setCentralWidget(None)
            if target_sub_win and sheet_widget:
                target_sub_win.setWidget(sheet_widget)
            sheet_frame.close()
        except Exception:
            pass

    target_sub_win.destroyed.connect(restore_on_document_delete)

```

```

        if not hasattr(FreeCADGui, '_mm_standalone_clones'):
            FreeCADGui._mm_standalone_clones = []
        FreeCADGui._mm_standalone_clones.append(sheet_frame)

        # Geometry positioning routine targeting secondary screen borders
layout safely
        self._apply_monitor_and_geometry(sheet_frame,
FreeCADGui.getMainWindow(), 1000, 700)
        sheet_frame.raise_()
        sheet_frame.activateWindow()

        FreeCAD.Console.PrintLog("MultiMonitor: Spreadsheet safely isolated and
detached onto secondary monitor.\n")
        except Exception as e:
            FreeCAD.Console.PrintError(f"MultiMonitor Error (Failed spreadsheet
window extraction): {str(e)}\n")

    def detach_techdraw_window(self, tab_index):
        """Extracts the native TechDraw vector drawing view panel and detaches it
into a top-level window frame safely."""
        if not self.mdi_area or tab_index >= len(self.mdi_area.subWindowList()):
            return

        target_sub_win = self.mdi_area.subWindowList()[tab_index]
        target_sub_win.setFocus()

        gui_doc = FreeCADGui.activeDocument()
        if not gui_doc:
            return

        try:
            # Fetch the main TechDraw graphics scene viewer widget profile
            techdraw_widget = target_sub_win.widget()
            if not techdraw_widget:
                return

            doc_name = gui_doc.Document.Name

            # --- SECURE CONTAINER MOUNTING ---
            # Build a dedicated frame manager window to host the vector graphics
sheet layout
            td_frame = QtWidgets.QMainWindow(None, QtCore.Qt.Window |
QtCore.Qt.CustomizeWindowHint | QtCore.Qt.WindowMinMaxButtonsHint |
QtCore.Qt.WindowCloseButtonHint)
            td_frame.setWindowTitle(f"TechDraw Page View -
{target_sub_win.windowTitle()}")
            td_frame.setStyleSheet("QMainWindow { background-color: #2b2b2b; }")

            # Embed the original vector scene widget directly as the central
workspace of the external window
            td_frame.setCentralWidget(techdraw_widget)
            techdraw_widget.show()

            # --- ELEGANT VISUAL PLACEHOLDER MASK ---
            # Inject a lightweight container card inside FreeCAD's native MDI to
satisfy the layout engine and Ribbon UI
            placeholder_card = QtWidgets.QWidget()
            placeholder_card.setStyleSheet("background-color: #2b2b2b;")
            card_layout = QtWidgets.QVBoxLayout(placeholder_card)

```

```

# Upscaled the monitor icon using HTML tags wrapper inside TechDraw
courtesy card
    lbl_notice = QtWidgets.QLabel("<span style='font-size:
36px;'>🖥️</span><br><br>The TechDraw drawing page has been moved to an external
window.", placeholder_card)
    lbl_notice.setAlignment(QtCore.Qt.AlignCenter)
    lbl_notice.setStyleSheet("color: #aaaaaa; font-size: 13px; font-weight:
bold;")

    # lbl_notice = QtWidgets.QLabel("🖥️ The TechDraw drawing page has been
moved to an external window.", placeholder_card)
    # lbl_notice.setAlignment(QtCore.Qt.AlignCenter)
    # lbl_notice.setStyleSheet("color: #aaaaaa; font-size: 13px; font-
weight: bold;")
    card_layout.addWidget(lbl_notice)

# Assign the temporary text card placeholder into FreeCAD's open MDI
sub-window framework
    target_sub_win.setWidget(placeholder_card)

# Disconnect any old conflicting listeners to avoid race conditions
loops
    try:
        target_sub_win.destroyed.disconnect()
    except Exception:
        pass

# --- INTEGRATED LIFECYCLE CONNECTIONS ---
# Hook 1: If the user closes the main document or the native tab, close
the external TechDraw window automatically
    def handle_techdraw_deletion():
        try:
            td_frame.close()
        except Exception:
            pass
    target_sub_win.destroyed.connect(handle_techdraw_deletion)

# Hook 2: Overriding the close behavior of the external window to
ensure it triggers the tab closure too
    def handle_external_close(event):
        try:
            # Return window control and sever widget links to prevent
cyclic deletion crashes in Qt
            td_frame.setCentralWidget(None)
            if target_sub_win:
                try:
                    target_sub_win.destroyed.disconnect()
                except Exception:
                    pass
            # Return the original widget to FreeCAD right before
closing to satisfy the C++ destructor
            target_sub_win.setWidget(techdraw_widget)
            # Let FreeCAD natively close and clear the drawing tab
completely from the workspace
            target_sub_win.close()
        except Exception:
            pass

```

```

        # Unregister from global tracking array memory structures
        if hasattr(FreeCADGui, '_mm_standalone_clones') and
FreeCADGui._mm_standalone_clones:
            if td_frame in FreeCADGui._mm_standalone_clones:
                FreeCADGui._mm_standalone_clones.remove(td_frame)
            event.accept()

        # Override the closeEvent method of our fresh window instance
dynamically via lambda assignment
        td_frame.closeEvent = handle_external_close

        if not hasattr(FreeCADGui, '_mm_standalone_clones'):
            FreeCADGui._mm_standalone_clones = []
            FreeCADGui._mm_standalone_clones.append(td_frame)

        # Render the standalone frame window onto the secondary screen geometry
coordinates safely
        td_frame.showNormal()
        self._apply_monitor_and_geometry(td_frame, FreeCADGui.getMainWindow(),
1100, 800)
        td_frame.raise_()
        td_frame.activateWindow()

        FreeCAD.Console.PrintLog("MultiMonitor: TechDraw page view successfully
projected onto secondary monitor.\n")
        except Exception as e:
            FreeCAD.Console.PrintError(f"MultiMonitor Error (Failed TechDraw page
extraction): {str(e)}\n")

    def _apply_monitor_and_geometry(self, window_instance, fallback_mw, default_w,
default_h):
        """Unified geometric calculations module targeting multi-monitor window
positioning using PySide."""
        if hasattr(FreeCADGui, '_mm_win_v26') and FreeCADGui._mm_win_v26:
            target_frame = FreeCADGui._mm_win_v26
        else:
            target_frame = fallback_mw
        if target_frame.windowHandle() and target_frame.windowHandle().screen():
window_instance.windowHandle().setScreen(target_frame.windowHandle().screen())
        else:
            target_screen =
QtWidgets.QApplication.screenAt(target_frame.geometry().center())
            if target_screen:
window_instance.windowHandle().setScreen(target_screen)
            frame_geo = target_frame.geometry()
            x_pos = frame_geo.x() + (frame_geo.width() - default_w) // 2
            y_pos = frame_geo.y() + (frame_geo.height() - default_h) // 2
            window_instance.setGeometry(x_pos, y_pos, default_w, default_h)

class MultiMonitorWindow6(QtWidgets.QMainWindow):
    @staticmethod
    def move_document_to_isolated_window_static(tab_index, window_class):
        """Static bridge to safely trigger the extraction logic from the context
menu event."""
        # Creiamo un'istanza al volo della classe del filtro per usare la logica di
movimento esistente
        dummy_filter = TabContextMenuFilter(None, window_class)
        dummy_filter.move_document_to_isolated_window(tab_index)

```

```

def __init__(self, is_standalone_doc=False):
    super(MultiMonitorWindow6, self).__init__(None, QtCore.Qt.Window)
    self.settings = QtCore.QSettings("FreeCAD", "MultiMonitor")
    self.central_widget = QtWidgets.QWidget()
    self.setCentralWidget(self.central_widget)

    self._extracted_views = []
    self._tree_view_data = None

    if not is_standalone_doc:
        self.setWindowTitle("FreeCAD - Secondary Monitor Layout")
        self.resize(1200, 800)

        self.main_layout = QtWidgets.QHBoxLayout(self.central_widget)
        self.main_layout.setContentsMargins(2, 2, 2, 2)

        self.horizontal_splitter = QtWidgets.QSplitter(QtCore.Qt.Horizontal,
self.central_widget)
        self.main_layout.addWidget(self.horizontal_splitter)

        self.right_vertical_splitter = QtWidgets.QSplitter(QtCore.Qt.Vertical,
self.horizontal_splitter)
        self.horizontal_splitter.addWidget(self.right_vertical_splitter)

        # INCREASED DELAY: Give FreeCAD 800ms to stabilize the core C++ dock
state before extraction
        QtCore.QTimer.singleShot(800, self.extract_widgets)
    else:
        self.setWindowTitle("FreeCAD - Detached Document Canvas")
        self.resize(1200, 800)

        local_layout = QtWidgets.QHBoxLayout(self.central_widget)
        local_layout.setContentsMargins(2, 2, 2, 2)

        self.document_tabs = QtWidgets.QTabWidget(self.central_widget)
        local_layout.addWidget(self.document_tabs)

        if self.settings.contains("standalone_geometry"):
            self.restoreGeometry(self.settings.value("standalone_geometry"))

    def extract_widgets(self):
        """Locates FreeCAD's native dock panels, isolates them, and loads splitters
layout configurations securely."""
        mw = FreeCADGui.getMainWindow()
        if not mw:
            return

        # Force Qt to process any pending UI refresh cycles before we alter dock
frames
        QtWidgets.QApplication.processEvents()

        dock_combo = None
        dock_selection = None
        dock_python = None
        dock_report = None

        # Deep scan active dock widgets inside FreeCAD's core shell
        for dock in mw.findChildren(QtWidgets.QDockWidget):

```

```

        obj_name = dock.objectName().lower()
        win_title = dock.windowTitle().lower()

        if "property" in obj_name or "combo" in obj_name or "property" in
win_title or "combo" in win_title:
            dock_combo = dock
        elif "select" in obj_name or "select" in win_title or "sel" in
obj_name:
            dock_selection = dock
        elif "python" in obj_name or "console" in obj_name or "python" in
win_title or "console" in win_title:
            dock_python = dock
        elif "report" in obj_name or "output" in obj_name or "report" in
win_title or "output" in win_title:
            dock_report = dock

    # 1. Mount Property/Combo View into the primary left column layout slot
    if dock_combo:
        mw.removeDockWidget(dock_combo)
        dock_combo.setParent(None)
        dock_combo.setFloating(False)
        self.horizontal_splitter.insertWidget(0, dock_combo)
        dock_combo.show()
        dock_combo.update()

    # 2. Mount Selection View into the central column layout slot
    if dock_selection:
        mw.removeDockWidget(dock_selection)
        dock_selection.setParent(None)
        dock_selection.setFloating(False)
        self.horizontal_splitter.insertWidget(1, dock_selection)
        dock_selection.show()
        dock_selection.update()

    # 3. Mount Python Console into the top slot of the right column vertical
splitter
    if dock_python:
        mw.removeDockWidget(dock_python)
        dock_python.setParent(None)
        dock_python.setFloating(False)
        self.right_vertical_splitter.addWidget(dock_python)
        dock_python.show()
        dock_python.update()

    # 4. Mount Report/Output View into the bottom slot of the right column
vertical splitter
    if dock_report:
        mw.removeDockWidget(dock_report)
        dock_report.setParent(None)
        dock_report.setFloating(False)
        self.right_vertical_splitter.addWidget(dock_report)
        dock_report.show()
        dock_report.update()

    # Force layout recalculation before loading sizes into the splitters engine
    QtWidgets.QApplication.processEvents()

    # --- SESSION LAYOUT RESTORE ENGINE ---
    saved_geometry = self.settings.value("window_geometry")

```



```

        if saved_geometry:
            self.restoreGeometry(saved_geometry)

        saved_horiz_splitter = self.settings.value("horiz_splitter_sizes")
        if saved_horiz_splitter:
            self.horizontal_splitter.setSizes([int(x) for x in
saved_horiz_splitter])

        saved_vert_splitter = self.settings.value("vert_splitter_sizes")
        if saved_vert_splitter:
            self.right_vertical_splitter.setSizes([int(x) for x in
saved_vert_splitter])

        # Ultimate visual anchor refresh
        self.update()

    def closeEvent(self, event):
        """Intercepts the window close event to hide the window instead of
destroying it, preserving widgets."""
        is_primary_addon_win = hasattr(self, 'right_vertical_splitter')

        # --- SESSION LAYOUT SAVE ENGINE ---
        if is_primary_addon_win:
            self.settings.setValue("window_geometry", self.saveGeometry())
            self.settings.setValue("horiz_splitter_sizes",
self.horizontal_splitter.sizes())
            self.settings.setValue("vert_splitter_sizes",
self.right_vertical_splitter.sizes())
        else:
            self.settings.setValue("standalone_geometry", self.saveGeometry())

        # If it is the main secondary monitor window, hide it and ignore the
destruction request
        if is_primary_addon_win:
            event.ignore() # Prevent Qt from destroying the window reference
            self.hide()    # Safely hide the window while keeping all extracted
widgets alive inside
            FreeCAD.Console.PrintMessage("MultiMonitor: Window hidden successfully,
preserving panel state.\n")
            return

        if hasattr(FreeCADGui, '_mm_isolated_windows') and self in
FreeCADGui._mm_isolated_windows:
            FreeCADGui._mm_isolated_windows.remove(self)

        event.accept()

class MultiMonitorAddon:
    """Core addon coordinator responsible for hooks injection and event lifecycle
tracking using PySide."""
    def __init__(self):
        self.filter_instance = None
        self.active_tabbar = None
        # Setup continuous monitoring loop for active tab swaps or tab recreations
        self.lifecycle_timer = QtCore.QTimer()
        self.lifecycle_timer.setInterval(2000) # Every 2 seconds
        self.lifecycle_timer.timeout.connect(self.insinuated_tabbar_search)

    def Setup(self):

```



```

        FreeCADGui._mm_win_v26.raise_()
        FreeCADGui._mm_win_v26.activateWindow()
        FreeCAD.Console.PrintMessage("MultiMonitor: Existing layout window
revealed and brought to front.\n")
        return
    except Exception:
        # Fallback if the reference became corrupted or dropped by the C++
engine
        FreeCADGui._mm_win_v26 = None

    if hasattr(FreeCADGui, '_mm_isolated_windows') and
FreeCADGui._mm_isolated_windows:
        for win in list(FreeCADGui._mm_isolated_windows):
            try:
                win.close()
            except Exception:
                pass
        FreeCADGui._mm_isolated_windows = []

    # Fallback/Initial Boot: Instantiate a completely fresh main secondary monitor
window
    FreeCADGui._mm_win_v26 = MultiMonitorWindow6()
    FreeCADGui._mm_win_v26.show()
    FreeCADGui._mm_win_v26.raise_()
    FreeCADGui._mm_win_v26.activateWindow()

class MultiMonitorMenuCommand:
    """Native FreeCAD Command class to handle action triggering safely."""
    def GetResources(self):
        return {
            'MenuText': "MultiMonitor Manager",
            'ToolTip': "Open or restore the multi-monitor interface panels"
        }

    def Activated(self):
        open_window()

def inject_button_into_statusbar():
    """Uses PySide to inject a dedicated button directly into FreeCAD's native
status bar safely."""
    try:
        mw = FreeCADGui.getMainWindow()
        if not mw:
            return

        status_bar = mw.statusBar()
        if not status_bar:
            return

        # Guard clause: Prevent injecting multiple duplicate buttons into the
layout
        if status_bar.findChild(QtWidgets.QPushButton, "btn_multimonitor_manager"):
            return

        # Create a clean, lightweight push button utilizing pure PySide styling
        btn = QtWidgets.QPushButton("🖥️ MultiMonitor", status_bar)
        btn.setObjectName("btn_multimonitor_manager")

        # Apply a subtle flat layout style that fits seamlessly into FreeCAD's

```

theme

```
    btn.setStyleSheet("""
        QPushButton {
            background-color: transparent;
            border: 1px solid #555555;
            border-radius: 3px;
            padding: 1px 6px;
            color: #aaaaaa;
            font-size: 11px;
            font-weight: bold;
        }
        QPushButton:hover {
            background-color: #3b3b3b;
            color: #ffffff;
            border-color: #888888;
        }
        QPushButton:pressed {
            background-color: #2b2b2b;
        }
    """)

    btn.setToolTip("Open or restore the multi-monitor interface panels")

    # Connect the button click event directly to our safe open_window factory
    btn.clicked.connect(open_window)

    # Insert the button onto the right side of the native status bar layout
    status_bar.addPermanentWidget(btn)
    FreeCAD.Console.PrintLog("MultiMonitor: Status bar trigger widget
successfully mounted.\n")

    except Exception as e:
        FreeCAD.Console.PrintWarning(f"MultiMonitor Status Bar Notice (Widget
injection bypassed): {str(e)}\n")

# ENVIRONMENT REGISTRATION ENTRY POINT
if FreeCADGui.getMainWindow():
    try:
        # Register the command inside FreeCAD's internal registry for macro
        execution compliance
        FreeCADGui.addCommand("MultiMonitor_Manager", MultiMonitorMenuCommand())
    except Exception:
        pass

    # Launch the continuous background monitoring loop for native document tabs
    if not hasattr(FreeCADGui, '_mm_addon_instance'):
        FreeCADGui._mm_addon_instance = MultiMonitorAddon()
        FreeCADGui._mm_addon_instance.Setup()

    # Asynchronously inject the status bar button to allow FreeCAD UI sub-
    structures to settle
    QtCore.QTimer.singleShot(1500, inject_button_into_statusbar)

    # Launch the window independently to instantly restore secondary monitor
    geometry settings
    QtCore.QTimer.singleShot(500, open_window)
```